

SSH Threat Detection & Firewall Response Lab

Complete Step-by-Step Command Guide

This guide consolidates the commands used throughout the laboratory into one practical reference. Each command has a short explanation so the project can be reproduced and understood without relying on the GitHub documentation alone.

Lab Environment

System	Role	IP used in the lab
Ubuntu Server	Monitored server / SSH / UFW / logs	192.168.10.100
Kali Linux	Security testing machine	192.168.10.101

1. Network & System Preparation

1.1 — Check the Ubuntu network configuration

Confirms the server IP before troubleshooting SSH or firewall issues.

```
ip -br addr
```

1.2 — Check the current user

Confirms the Ubuntu username so paths are not guessed.

```
whoami
```

1.3 — Go to the home directory

Uses the home directory as a convenient location for the detection script.

```
cd ~
```

1.4 — List files

Checks whether the detection script is present.

```
ls -l
```

2. SSH Service Configuration & Verification

2.1 — Check SSH service status

Verifies whether OpenSSH is running.

```
sudo systemctl status ssh
```

2.2 — Start SSH if necessary

Starts SSH if it is not already running.

```
sudo systemctl start ssh
```

2.3 — Enable SSH at boot

Makes SSH start automatically after reboot.

```
sudo systemctl enable ssh
```

2.4 — Confirm SSH port

Checks whether the server is listening on TCP 22.

```
sudo ss -tulpn | grep :22
```

2.5 — Test SSH from Kali

Tests remote connectivity to Ubuntu.

```
ssh ola@192.168.10.100
```

3. UFW Firewall Configuration

3.1 — Check UFW before changes

Reviews existing firewall rules before changes.

```
sudo ufw status numbered
```

3.2 — Allow SSH

Permits SSH access when required.

```
sudo ufw allow 22/tcp
```

3.3 — Enable UFW

Activates the firewall after reviewing rules.

```
sudo ufw enable
```

3.4 — Review numbered rules

Makes individual rules easy to identify.

```
sudo ufw status numbered
```

3.5 — Delete a specific rule

Deletes the current rule number; always check numbering first.

```
sudo ufw delete <rule-number>
```

3.6 — Deny SSH from the test source

Demonstrates the controlled containment response.

```
sudo ufw deny from 192.168.10.101 to any port 22 proto tcp
```

3.7 — Verify containment

Confirms the deny rule is active.

```
sudo ufw status numbered
```

4. Controlled SSH Authentication Event

The lab used a controlled invalid-user authentication event to create evidence in Ubuntu's authentication log. It was not a sustained brute-force attack.

4.1 — Generate the controlled SSH event

Creates a failed authentication event for investigation.

```
ssh wronguser@192.168.10.100
```

4.2 — Check basic reachability

Helps separate network problems from SSH problems.

```
ping -c 4 192.168.10.100
```

5. SSH Log Investigation

5.1 — View recent authentication events

Shows recent authentication activity.

```
sudo tail -n 50 /var/log/auth.log
```

5.2 — Filter SSH daemon events

Focuses on sshd-generated events.

```
sudo grep "sshd" /var/log/auth.log | tail -20
```

5.3 — Find invalid users

Finds attempts using usernames that do not exist.

```
sudo grep "Invalid user" /var/log/auth.log
```

5.4 — Find recent invalid users

Limits the result to recent events.

```
sudo grep "Invalid user" /var/log/auth.log | tail -10
```

5.5 — Find failed passwords

Identifies failed SSH password authentication.

```
sudo grep "Failed password" /var/log/auth.log
```

5.6 — Find recent failed passwords

Limits failed-password results to recent events.

```
sudo grep "Failed password" /var/log/auth.log | tail -10
```

5.7 — Combined SSH detection search

Searches for both key indicators.

```
sudo grep -Ei "Failed password|Invalid user" /var/log/auth.log | tail -10
```

6. Create the Detection Script

The script searches the authentication log for simple SSH indicators and writes recent matches to a separate alert file.

6.1 — Create the script

Opens a new Bash script in the Ubuntu home directory.

```
nano ~/ssh_alart.sh
```

Script content:

```
#!/bin/bash

# SSH Threat Detection Script
# This script checks the Ubuntu authentication log
# for failed SSH authentication attempts and invalid users.

LOG_FILE="/var/log/auth.log"
ALERT_FILE="$HOME/security_alarts.log"

echo "===== " >> "$ALERT_FILE"
echo "SSH Security Check - $(date)" >> "$ALERT_FILE"
echo "===== " >> "$ALERT_FILE"

echo "[+] Failed SSH authentication attempts:" >> "$ALERT_FILE"
grep -Ei "Failed password|Invalid user" "$LOG_FILE" | tail -10 >> "$ALERT_FILE"

echo "" >> "$ALERT_FILE"
echo "[+] SSH security check completed." >> "$ALERT_FILE"
echo "" >> "$ALERT_FILE"
```

6.2 — Make the script executable

Allows the script to run directly.

```
chmod +x ~/ssh_alart.sh
```

6.3 — Run the script manually

Performs the first functional test.

```
~/ssh_alart.sh
```

6.4 — View the alert log

Checks whether the script wrote detection results.

```
cat ~/security_alarts.log
```

6.5 — Trace execution

Shows commands executed by Bash for troubleshooting.

```
bash -x ~/ssh_alart.sh
```

7. Cron Automation

7.1 — Open the user's Cron table

Opens the scheduler configuration.

```
crontab -e
```

7.2 — Schedule the detection script

Runs the detection script every minute in the lab.

```
*/1 * * * * /home/ola/ssh_alart.sh
```

7.3 — Verify the Cron entry

Confirms the task was saved.

```
crontab -l
```

7.4 — Check scheduled output

Confirms that the alert file is receiving output.

```
cat ~/security_alerts.log
```

8. IOC Investigation

8.1 — Search invalid-user indicators

Extracts relevant lines for IOC analysis.

```
sudo grep "Invalid user" /var/log/auth.log | tail -10
```

8.2 — Search failed-password indicators

Extracts authentication failures for correlation.

```
sudo grep "Failed password" /var/log/auth.log | tail -10
```

8.3 — Review recent SSH context

Provides a wider view of SSH activity.

```
sudo grep "sshd" /var/log/auth.log | tail -20
```

9. Firewall Containment & Validation

9.1 — Apply controlled containment

Restricts SSH from the known Kali test address.

```
sudo ufw deny from 192.168.10.101 to any port 22 proto tcp
```

9.2 — Confirm active rules

Verifies the containment rule.

```
sudo ufw status numbered
```

9.3 — Test SSH from Kali

Checks whether the response affects the intended connection.

```
ssh ola@192.168.10.100
```

9.4 — Validate TCP 22 with Nmap

Checks how port 22 appears from Kali.

```
nmap -p 22 192.168.10.100
```

10. Troubleshooting Commands

10.1 — Check interfaces

Useful when diagnosing network problems.

```
ip -br addr
```

10.2 — Test reachability

Checks basic IP connectivity.

```
ping -c 4 192.168.10.100
```

10.3 — Check SSH service

Separates service problems from network problems.

```
sudo systemctl status ssh
```

10.4 — Check port 22

Confirms whether something is listening on SSH port 22.

```
sudo ss -tulpn | grep :22
```

10.5 — Review firewall rules

Useful whenever connectivity changes after firewall edits.

```
sudo ufw status numbered
```

10.6 — Check Cron

Confirms the scheduled task exists.

```
crontab -l
```


11. GitHub Project Organization

The repository was organized with the README as the project overview, Documentation for detailed write-ups, and Scripts for the actual Bash artifact.

```
SSH-Threat-Detection-Firewall-Response/
├── README.md
├── Documentation/
│   ├── 01-Network-Configuration.md
│   ├── 02-SSH-Configuration.md
│   ├── 03-Firewall-Configuration.md
│   ├── 04-SSH-Attack-Simulation.md
│   ├── 05-SSH-Log-Investigation.md
│   ├── 06-SSH-Detection-Script.md
│   ├── 07-Cron-Automation.md
│   ├── 08-IOC-Investigation.md
│   ├── 09-Firewall-Containment.md
│   ├── 10-Validation-and-Testing.md
│   ├── 11-Challenges-and-Troubleshooting.md
│   ├── 12-Conclusion.md
│   └── 13-MITRE-ATT&CK-Mapping.md
└── Scripts/
    └── ssh_alart.sh
```

12. MITRE ATT&CK; Context

T1021.004 (Remote Services: SSH) directly relates to the SSH service used in the lab. T1110 (Brute Force) is relevant to the authentication-failure detection context, but the lab used a controlled invalid-user event rather than a sustained brute-force attack.

13. Project Flow

```
Network Configuration
  ↓
SSH Configuration
  ↓
UFW Configuration
  ↓
Controlled SSH Event
  ↓
/var/log/auth.log
  ↓
Detection Script
  ↓
Cron Automation
  ↓
IOC Investigation
  ↓
Firewall Containment
  ↓
SSH / Nmap Validation
```

Important Notes

1. Replace the example username 'ola' with the actual Ubuntu username if it differs in your lab.
2. Check 'sudo ufw status numbered' before deleting a firewall rule because rule numbers can change.
3. Do not describe a single controlled invalid-user event as proof of a brute-force attack.
4. The commands are intended for the isolated laboratory environment documented in the project.